

Control de Flujo

Introducción

El **control de flujo** es uno de los temas centrales de un lenguaje de programación.

La manera como se incorpore en el lenguaje determina el orden de ejecución de un programa.

Los mecanismos del lenguaje utilizados para especificar orden de ejecución, pueden ordenarse en categorías:

- Secuencia
- Selección
- Iteración
- Abstracción procedural
- Recursión
- Concurrencia
- Manejo de excepciones o especulación
- No determinismo

Evaluación de expresiones

Las expresiones son una forma fundamental de especificar computación en un lenguaje de programación.

Además de la forma de las mismas (sintaxis: BNF), su significado (semántica) es crucial.

La semántica, esta gobernada por su forma de evaluación.

Es necesario estar conscientes de los ordenes de evaluación de operandos y operadores

Una expresión consiste generalmente de un objeto simple (contante literal, constante nombrada o variable) o un operador o una función aplicada a una colección de operandos o argumentos (cada uno de los cuales define una expresión)

Expresiones

Las expresiones constan de:

- Operadores
- Operandos
- Paréntesis
- Llamadas a función

Los operadores pueden ser:

- Unarios: tienen sólo un operando
- Binarios: tienen dos operandos

Algunos lenguajes definen sus operadores como “*syntactic sugar*” para conseguir un aspecto mas normal. Así por ejemplo $a+b$ es internamente $+(a,b)$

Expresiones Aritméticas

La mayoría de los lenguajes imperativos adoptan una notación infija para los operadores, pero hay otras opciones:

- Prefija
 - $op\ a\ b$
 - $op(a,b)$
 - $(op\ a\ b)$
- Infija
 - $a\ op\ b$
- Posfija
 - $a\ b\ op$

La mayoría de los lenguajes imperativos utilizan notación infija para los operadores binarios y prefija para los unarios.

Expresiones

Aspectos de diseño:

- Reglas de precedencia de operadores
- Reglas de asociatividad de operadores
- Orden de evaluación de los operandos
- Evaluación de efectos colaterales de un operador
- Sobrecarga de un operador
- Modo mixto de expresiones
- Errores en las expresiones

Control

Asignación

En el paradigma **funcional**, el cómputo consiste en evaluar expresiones. Las expresiones resultan ser los bloques básicos del lenguaje.

En el paradigma **imperativo**, el cómputo consiste en una serie ordenada de cambios (de los valores de las variables en memoria). Las **asignaciones** resultan el medio principal para realizar esos cambios.

El constructor de una lenguaje de programación produce un **efecto colateral**, si influencia al resto de la computación de otra manera que no sea a través de su retorno

Asignación

A simple vista, la asignación parece una operación directa.

A nivel implementación, sin embargo, se producen diferencias sutiles que pueden dar lugar a diferentes tipos de asignación.

Estas diferencias a simple vista pueden resultar invisibles (no afectan el comportamiento en programas simples)

En C:

$$d = a$$

El lado derecho hace
referencia al valor de a

$$a = b + c$$

El lado izquierdo hace
referencia a la locación de a

Asignación

Se puede definir la operación de asignación como:

- Computar el valor del lado izquierdo (**l-value**)
- Computar el valor de la expresión del lado derecho (**r-value**)
- Asignar el valor computado del lado derecho al computado como objeto de dato del lado derecho
- Retornar el valor computado como valor del lado derecho como resultado de la asignación

l-value: las expresiones que denotan locaciones
r-value: las expresiones que denotan valores

La semántica de la operación de asignación puede pensarse como:

- Copia de la referencia
- Dereferenciamiento implícito

Operadores de asignación combinada

Para evitar situaciones de asignación triviales, como

```
a = a+1
```

algunos lenguajes proveen operadores abreviados, así la sentencia anterior puede escribirse como:

```
a++
```

Asignación

La operación de asignación puede pensarse como:

- una instrucción (no retorna ningún valor)
- una expresión

Instrucción	Pascal, Ada, Algol, Fortran, PL/1
Expresión	C, Java, APL, ML, Snobol 4

Así por ejemplo, en C uno puede escribir la siguiente asignación:

$$C = (B = 2)$$

La cual se interpreta como se le asigna el valor de 2 a B. El resultado de la expresión $B=2$, es decir 2, se la asigna a C. Por lo cual se le asigna 2 a C

Asignación

La asignación puede en algunas circunstancias darse en forma implícita. Generalmente esto sucede en:

- La inicialización en las declaraciones
- Pasaje de parámetros

Ordenamiento

Las reglas **precedencia** y **asociatividad** establecen el orden en los operadores infijos de una expresión se aplican pero no determinan el orden de evaluación.

$$a - f(b) - c * d$$

Define cómo se resuelve el orden de evaluación frente a operadores del mismo nivel de precedencia.

- Regla típica: evaluación de izquierda a derecha

Hay dos razones por las que el orden es fundamental:

- Efectos colaterales
- Optimización de código

Si ninguno de los operandos u operadores tiene efectos colaterales, entonces el orden de evaluación es irrelevante

Evaluación de expresiones

Nos resta ver las posibles forma de evaluar las expresiones.

Los tipos fundamentales son:

1. Evaluación Estricta
2. Evaluación No Estricta

Evaluación Estricta: los argumentos de una función o los operandos de la expresión son siempre completamente evaluados antes de la aplicación de la función u operador.

Evaluación No Estricta: los argumentos de una función o los operandos de la expresión no son evaluados a menos que sean utilizados en el cuerpo de la función o sean necesarios para determinar el valor de la expresión.

Evaluación estricta

La evaluación estricta es la tradicionalmente utilizada por los lenguajes de programación.

Es también conocida como:

- Evaluación de orden aplicativo (evalúa de izq. a der.)
- Evaluación ansiosa

En este tipo de evaluación una expresión se evalúa al momento de establecer la ligadura con una variable.

Los lenguajes imperativos la utilizan casi siempre (debido a que el orden de ejecución está definido implícitamente por la organización del código)

Evaluación estricta

Ventajas:

- Elimina la necesidad de planificar
- El programador puede determinar el orden de ejecución

Desventajas:

- Fuerza la evaluación de expresiones que pueden no ser necesarias en tiempo de ejecución.
- Puede demorar la evaluación de expresiones cuyo valor se requiere en forma más inmediata.
- Fuerza al programador a organizar el código fuente para garantizar una ejecución óptima.

Evaluación no estricta

Cuando se implementa se denomina **evaluación de orden normal**.

Características:

- La evaluación se reduce por la izquierda
- Las funciones se aplican antes de evaluar sus argumentos.

Se puede especializar en distintas versiones:

- **Evaluación perezosa**: agrega la memorización de los argumentos evaluados.
- **Evaluación corto-circuito** (expresiones booleanas): se retorna el valor de la expresión tan pronto como se pueda determinar el valor de verdad sin ambigüedad

Evaluación de orden normal: Ejemplo

Ejemplo: Considere la siguiente llamada sumar(2+3)

```
sumar(x):
```

```
    y = 2+3 * 2
```

```
    z = 2+3 + 10
```

El resultado, se obtiene sustituyendo `x` con la expresión `2+3`.

La expresión `2+3` y no el valor 5, es pasado como el de valor de `x` en la función.

Evaluación perezosa: Ejemplo

Ejemplo: Considere la siguiente llamada sumar(2+3)

```
sumar(x):
```

```
    y = 2+3 * 2
```

```
    z = 2+3 + 10
```

El resultado, se obtiene sustituyendo x con la expresión $2+3$, en la primera aparición de x dentro del cuerpo de sumar y por el valor de la expresión, 5 , en las futuras apariciones de x en el cuerpo de la función.

Evaluación perezosa: Ejemplo

Ejemplo en ML

```
fun g(x,y,z) = if x<2 then y+3 else z+6
```

Ejemplo en C

```
a=0: 0 ? b/a
```

Es interesante notar que solo el valor de **y** o el valor de **z** son requeridos para el cálculo del resultado, pero no ambos.

Ejemplo en Haskell

```
fibs = 0 : 1 : zipWith (+) fibs(tail fibs)
```

Esta función calcula una lista con todos los números Fibonacci.

“:” agrega un elemento a la lista

Tail: retorna una lista sin su primer elemento

zipWith crea una nueva lista tomando el elemento a la cabeza de cada lista, aplicando una función, + en este caso.

Evaluación corto-circuito: Ejemplo

Ejemplo en C

```
int a = 0; int b = 4
if (a != b && myfun(b) )
{
    do_something();
}
```

En este ejemplo, debido a la evaluación en corto-circuito, se garantiza que la llamada a `myfun(b)` nunca suceda. Esto es porque `a` se evalúa como falso

Otro ejemplo en C

```
int TieneTresCaracteresDeLongitud(const char *p)
{
    return (p != NULL) && (strlen(p) ==3);
}
```

Control de flujo no estructurado

Se provee a través saltos [condicionales](#) e [incondicionales](#).

Los primeros lenguajes cercanos al hardware de la máquina trabajaban con tipos básicos y con etiquetas o rótulo y saltos como instrucciones.

Aspectos de diseño:

Los rótulos son de suma importancia, es necesario establecer las restricciones sobre ellos.

- Constantes
- Expresiones sin cómputo en ejecución
- Expresiones

Transferencia de control

```
if E1
  then C1: goto L1
  else C2: goto L2
endif
```

```
L1: C3
      while E2 do C4
L2: C5
      goto L1
      C6
enddo
```

El uso de las instrucciones como el **goto** conducen a programas con diseño no estructurado. Haciendo muy difícil utilizar modelos de correctitud para los programas. En general sus ventajas no amortizan sus grandes desventajas.

Se puede demostrar que el uso de este tipo de instrucciones es superfluo ya que puede simularse fácilmente con **secuencias de control estructuradas**.

Control de flujo estructurado

El abandono de esta forma de control fue parte de determinar la programación estructurada.

En este entorno el control no estructurado se mantuvo, pero asociado a casos especiales y con condicionantes (no se puede transferir el control a cualquier lugar).

Opciones de alto nivel:

- **Break**: salida incondicional
- **Continue**: se utiliza en las iteraciones. El control salta al final del bucle, con lo que el contador se incrementa y comienza otra comprobación

Programación estructurada: diseño

Resumiendo, para obtener programas más fáciles de entender, verificar, corregir, modificar y re-verificar se debe enfatizar:

1. Organización jerárquica de la estructura del programa (uso de composición, alternación e iteración)
2. Uso de estructuras de control estructuradas (una entrada – una salida por instrucción)
3. Correspondencia entre el orden del texto del programa y el orden de la ejecución
4. Usar grupos de sentencias con propósito simple, aún cuando se requiera copia.

Estructuras de control estructuradas

Una estructura de control es una sentencia de control y una colección de sentencias cuya ejecución controla.

Las estructuras de control estructuradas son:

- Secuencia
- Condicional
- Iteración

Secuencia de control estructurado

Un **sentencia compuesta** es un secuencia de instrucciones que pueden ser tratadas como una sola sentencia en la construcción de sentencia más grandes.

Representa básicamente la composición

El orden en el que aparecen en el texto del programa es el orden en el cual son ejecutadas

Aspectos de diseño:

- Delimitadores
- Separadores y terminadores
- Bloques

Sentencia Condicional

Una **sentencia condicional** es aquella que expresa una alternativa entre dos o más sentencias.

El control de las alternativas es controlado por una condición, usualmente expresada a través de una expresión booleana.

La forma más común de condicional es la sentencia **if-then-else**.

Aspectos de diseño:

- Condición simple, doble o múltiple
- Instrucción simple, compuesta o secuencia
- Anidamiento
- Forma y tipo de las expresiones de control

Sentencias condicional doble

La forma general que adopta el condicional doble en los lenguajes de programación es:

```
If <condicion> then <instruccion> else <instruccion>
```

La instrucción puede ser simple o compuesta

Anidamiento de sentencias condicionales

La mayoría de los lenguajes permite anidar sentencias condicionales, lo que puede provocar algunos problemas semánticos (interpretación del programa resultante).

Así si se tienen dos `if` y un solo `else`, aparece el problema de determinar a que `if` corresponde el `else`.

- Convención: asociar el `else` al `if` más cercano: C, C++, C# y Java
- Uso de sentencias compuestas: Perl
- Uso de terminador de sentencia: `endif` en Ada

Sentencias condicional múltiple

La sentencia **condicional múltiple** permite la selección de una de una grupo no fijo de sentencias o grupos de sentencias.

Aspectos de diseño:

- Alternativas excluyentes o no
- Forma y tipo de las expresiones selectoras
- Como se especifica el segmento seleccionado
- Valores no representados
- Instrucción simple, compuesta o secuencia
- Flujo de ejecución cuando termina

Comandos con guarda: múltiple selección con if

Forma general:

if <expresión> → <sentencia> [“[”<expresión> → <sentencia>]* fi

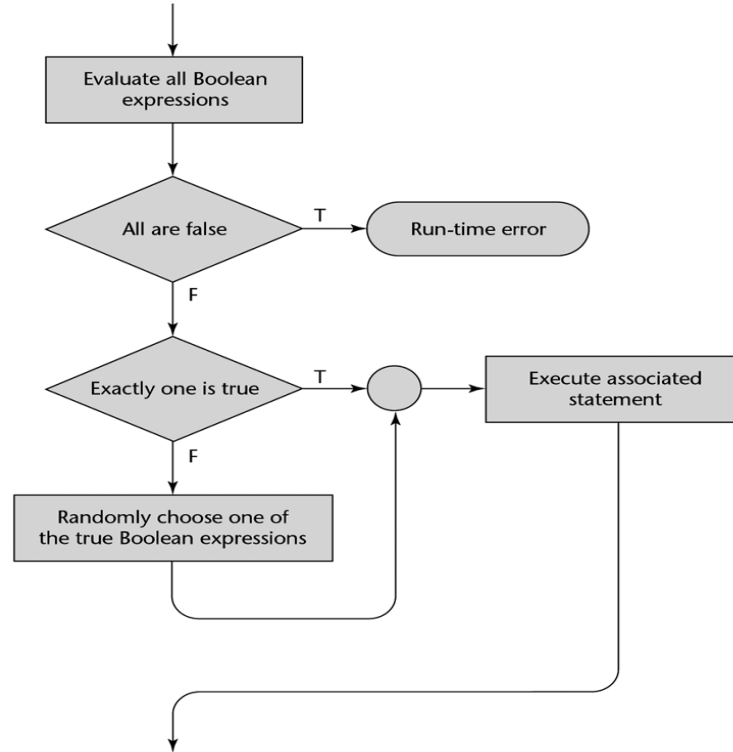
Semántica: cuando el constructor es alcanzado

1. Evaluar todas las expresiones booleanas
2. Si más de una es verdadera, elegir una en forma no-determinista
3. Si ninguna es verdadera, se produce un error en ejecución

Comandos con guarda: múltiple selección con if

Figure 8.1

Flowgraph of the approach used with Dijkstra's selector statement



Sentencia iterativa

La última de las estructuras de control estructuradas es la iteración

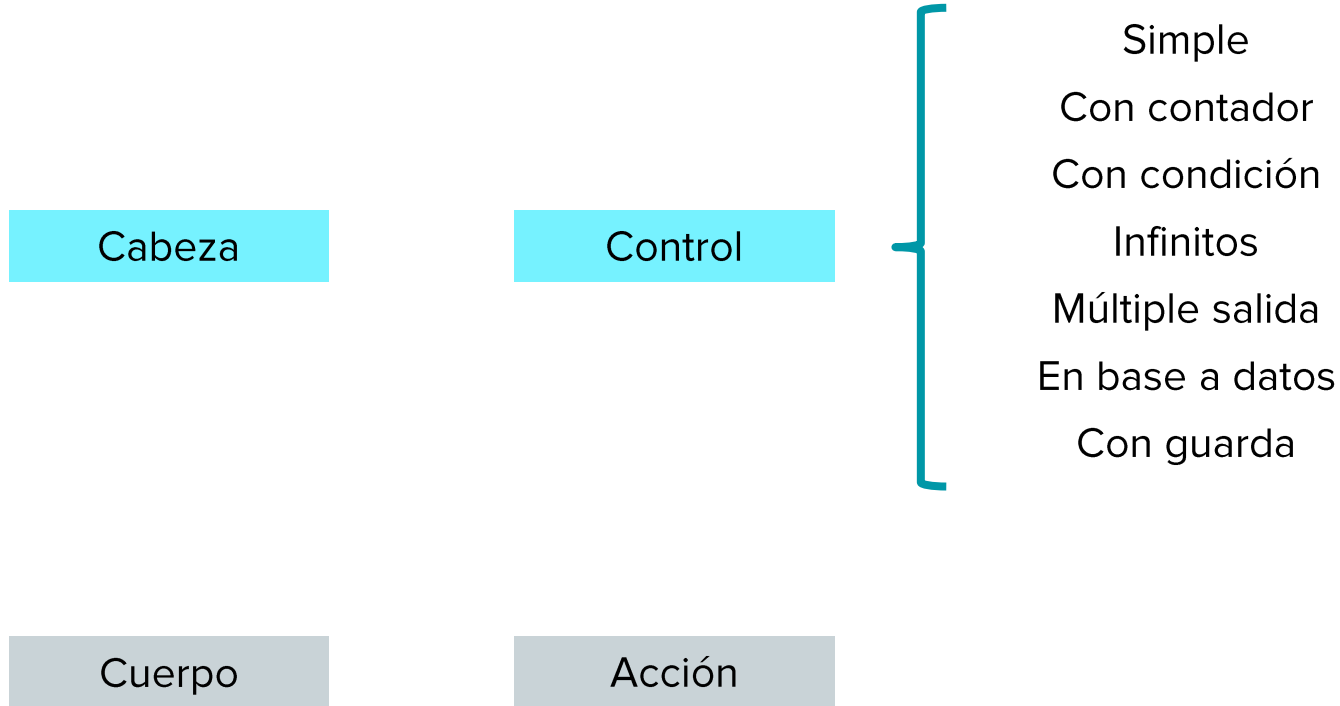
Una sentencia iterativa es aquella que cause que una sentencia o colección de sentencias se ejecute cero o más veces. Generalmente llamadas loop o ciclos

Los lenguajes que proveen facilidades para definir iteración conducen a programas menos extensos, flexibles, más simples de escribir y almacenar

Aspectos de diseño:

- ¿Cómo se controla la iteración?
- ¿Dónde debe aparecer el mecanismo de control en el constructor de ciclo?

Sentencia iterativa



Sentencia iterativa: iteración con contador

Aspectos de diseño:

- ¿Cuándo se evalúan las expresiones que controlan el bucle? ¿antes de comenzar el bucle o en cada iteración?
- ¿Cuál es el tipo y el alcance de la variable de control?
- ¿Cuál es el valor de la variable de control una vez que el bucle termina?
- ¿Puede modificarse el valor de la variable de control dentro del bucle? ¿altera la cantidad de iteraciones?

Sentencia iterativa: iteración en base de datos

Aspectos a tener en cuenta:

- El número de elementos de la estructura de datos controla la cantidad de iteraciones del ciclo
- El mecanismo de control es una llamada a una función iterador que retorna el próximo elemento en algún orden establecido (si es que hay un próximo elemento, sino termina)
- Las instrucciones como el **foreach** de C# iteran sobre los elementos de arreglos y otras colecciones.

Ejemplo en C++ y Java

```
x.start();
```

```
While X.more() { ... y = X.next() }
```

```
For (ptr = root; ptr != null; traverse(ptr))  
    {...}
```

Ejemplo en Pearl

```
Foreach $X (@arrayitem) {...}
```

En cada paso del ciclo, la variable escalar \$X tendrá el próximo elemento del arreglo @arrayitem

Sentencia iterativa: iteración con guarda

Propósito: soportar una nueva forma de programación que permita la verificación durante el desarrollo.

Resulta básico para dos mecanismos lingüísticos para la programación concurrente (Ada y CSP)

La idea básica es que si el orden de evaluación no es importante, el programa no debería especificar uno.

Forma del ciclo con guarda:

do <expresión> $\rightarrow$ <sentencia> [<expresión> $\rightarrow$ <sentencia>]* **od**

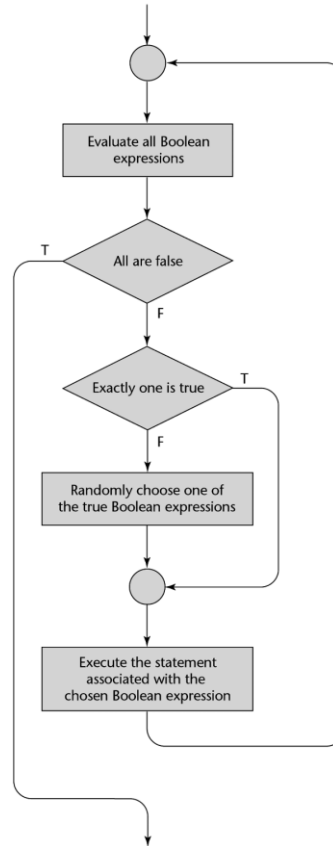
Semántica:

1. Evaluar la expresión (booleana en general)
2. Si más de una es verdadera, elegir una en forma no determinista y comenzar el ciclo nuevamente
3. Si ninguna es verdadera, salir del bucle

Sentencia iterativa: iteración con guarda

Figure 8.2

Flowgraph of the approach used with Dijkstra's loop statement



Iteradores

En general los ciclos con contador, tipo *for*, iteran sobre una secuencia numérica. Sin embargo es interesante poder hacerlo sobre los elementos de un conjunto cuyos elementos estén bien definidos (como las clases contenedoras o colecciones de los lenguajes orientados a objetos).

La idea del **iterador** es proveer esta funcionalidad.

Iteradores “reales” (true iterators)

En estos casos el iterador es una abstracción que se parece a un unidad que contiene una sentencia **yield** que produce el cambio en el valor del ciclo.

Algunos lenguajes que proveen *true iterator* son: Clu, Python, Ruby y C#

A diferencia de los iteradores de Java y C++, estos iteradores funcionan en base al contador del programa y las variables locales del entorno de ejecución.

Objetos Iteradores

Estos iteradores no utilizan la sentencia *yield*.

Se trata de un objeto ordinario o común de la programación orientada a objetos, que provee métodos para inicializar, generar el próximo valor del índice, testear su terminación. Entre llamadas, el estado del iterador debe mantenerse.

Los iteradores de Java y C++ estos objetos necesitan mantener el estado interno en forma de estructuras de datos explícitas.

En Java el uso se ve como en el siguiente ciclo

```
for (Integer i: myTree) { ...}
```

El iterador y su funcionamiento no se ven en la sintaxis.

Iteradores: Ejemplo

Java

```
Class TreeNode<T> implements Iterable<T> {  
    ...  
    private class Treeliterator implements Iterable<T> {  
        .....    }  
    ..... }  
    .....  
}
```

```
TreeNode<Integer> myTree = ....
```

```
for (Integer i: myTree) { .....}
```

```
for (Iterator<Integer> it= myTree.iterator();  
    it.hasNext();) { .....  
    ..... }
```

Python

```
def counter(low, high):  
    current = low  
    while current <= high:  
        yield current  
        current += 1
```

```
for c in counter(3,8):  
    print c
```

Recursión

Es un mecanismo de control que no requiere sintaxis adicional.

Aquellos lenguajes que proveen abstracciones procedurales y/o funcionales solo deben habilitar que las mismas tengan la posibilidad de llamarse a si mismas.

Esta manera de estructurar el control permite llegar en algunas oportunidades a códigos más elegantes.

No determinismo

Es un mecanismo en el cual la elección entre alternativas, queda deliberadamente sin especificar.

Esta característica se ve en algunos constructores para concurrencia, como los comandos con guarda.

¿Preguntas?
